



## Abschlussprojekt Angewandtes Maschinelles Lernen

Krystian Gluc - 12230903  
Cristian Grigore - 12312344  
Dominik Berghammer - 12316294  
Adrian Linder - 12243106

4. Februar 2026, Vienna

### Inhaltsverzeichnis

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Thema</b>                                                          | <b>2</b>  |
| 1.1      | Technologien                                                          | 2         |
| 1.2      | BERT                                                                  | 2         |
| <b>2</b> | <b>Das Ausfüllen von Lückentexten</b>                                 | <b>3</b>  |
| 2.1      | Einfaches Ausfüllen mithilfe von BERT                                 | 3         |
| 2.2      | Interessante Beobachtungen                                            | 4         |
| 2.3      | Verschiedene Modelle für verschiedene Themengebiete                   | 6         |
| 2.3.1    | Einleitung                                                            | 6         |
| 2.3.2    | Durchführung                                                          | 6         |
| 2.3.3    | Ergebnisse                                                            | 7         |
| 2.3.4    | Schlussfolgerung                                                      | 8         |
| <b>3</b> | <b>Finetune Beispiel mit BERT</b>                                     | <b>9</b>  |
| 3.1      | Dataset                                                               | 9         |
| 3.2      | Training                                                              | 11        |
| 3.3      | Auswertung                                                            | 12        |
| 3.4      | Informationsverlust                                                   | 14        |
| <b>4</b> | <b>Korrektur von Lückentexten</b>                                     | <b>15</b> |
| 4.1      | Erster Versuch mit BERT                                               | 15        |
| 4.1.1    | Datenset für Korrektur                                                | 15        |
| 4.1.2    | Korrektur mit BERT                                                    | 16        |
| 4.2      | Korrektur auf CLOTH Datenset                                          | 18        |
| 4.3      | Vergleich von verschiedenen Methoden zum Korrigieren von Lückentexten | 20        |

# 1 Thema

Nach unserem ersten Versuch allgemeiner angelegte Methoden zur Textgenerierung zu bewerten und von Scratch zu trainieren, haben wir uns auf eine spezifischere Aufgabe fokussiert: Masked-Language-Models und Lückentexte. Zu diesem Thema wurden dann drei verschiedene Aufgaben spezifiziert:

1. Das Ausfüllen von Lückentexten
2. Das Finetunen von MLMs
3. Das Korrigieren von Lückentexten

## 1.1 Technologien

Bei einer Recherche zu diesem Thema stößt man schnell auf den Begriff Masked-Language-Models. Eines der bekanntesten Modelle dazu ist BERT (Bidirectional Encoder Representations from Transformers). Es gibt auch zahlreiche andere Modelle, die für Lückentexte verwendet werden können. Einige Beispiele hierzu wären BART, RoBERTa, ALBERT, ELECTRA, MPNet wobei hinter vielen von diesen Modellen die BERT Architektur steckt. Wenn man solch einem Modell einen maskierten Text gibt, antwortet dieses mit dem Wort, das sich am wahrscheinlichsten hinter der Maske versteckt.

Bei dem Satz "[MASK] is the capital of France." würde so ein Modell zum Beispiel das Wort "Paris" vorhersagen.

## 1.2 BERT

BERT ist ein von Google entwickeltes Sprachmodell. Wie in [2] gezeigt sind *Masked LM* und *Next Sentence Prediction* die zwei Aufgaben die verwendet wurden, um BERT zu trainieren. Wobei *Masked LM* genau den Fall eines Lückentextes abdeckt. Das Model wurde hier mit Sätzen trainiert bei denen 15 % der Tokens des Satzes maskiert wurden.

Vor BERT waren die meisten Sprachmodelle *uni-directional*, was bedeutet das in dem Fall eines maskierten Textes nur Token vor der Lücke betrachtet werden. Ein bidirektionaler Encoder, wie der im Falle von BERT, betrachtet alle Token die vor und nach dem maskierten Wort stehen.

Dadurch wird auch ermöglicht, dass das vortrainierte BERT-Modell mit nur einem zusätzlichen Outputlayer *finetuned* werden kann.

## 2 Das Ausfüllen von Lückentexten

### 2.1 Einfaches Ausfüllen mithilfe von BERT

Eine der einfachsten Möglichkeiten, einen Lückentext automatisch auszufüllen, ist die Verwendung von BERT über die *transformers*-Bibliothek.

```
1 from transformers import pipeline
2
3 pipeline_bert = pipeline(
4     task="fill-mask",
5     model="bert-base-uncased",
6     device=0 # benutzt GPU statt CPU
7 )
8 pipeline_bert(
9     "Plants create [MASK] through a process [MASK] as photosynthesis.",
10    top_k=2 # gibt die Top 2 Resultate zurück
11 )
12 )
```

Nachdem man diesen Code ausführt, erhält man folgendes Resultat.

```
1 [
2   [{'score': 0.2662782669067383,
3     'token': 2943,
4     'token_str': 'energy',
5     'sequence': '[CLS] plants create energy through a process [MASK] as
6     photosynthesis. [SEP]'},
7   {'score': 0.10305079817771912,
8     'token': 4870,
9     'token_str': 'flowers',
10    'sequence': '[CLS] plants create flowers through a process [MASK] as
11    photosynthesis. [SEP]'}],
12
13   [{'score': 0.9980964064598083,
14     'token': 2124,
15     'token_str': 'known',
16     'sequence': '[CLS] plants create [MASK] through a process known as
17     photosynthesis. [SEP]'},
18   {'score': 0.001142986468039453,
19     'token': 2107,
20     'token_str': 'such',
21     'sequence': '[CLS] plants create [MASK] through a process such as
22     photosynthesis. [SEP]'}]
23 ]
```

Da der Lückentext zwei Lücken enthält, bekommt man eine Liste mit zwei Resultaten zurück. Dies sind jeweils die zwei Vorschläge für Lücke 1 und Lücke 2.

Anhand des Scores ist sofort ersichtlich, dass sich das Modell bei der zweiten Lücke deutlich sicherer fühlt als bei der ersten Lücke.

Das Modell schlägt uns also den folgenden ausgefüllten Lückentext vor:

*Plants create **energy** through a process **known** as photosynthesis.*

## 2.2 Interessante Beobachtungen

Bei der Erprobung der verschiedenen Modelle kamen einige interessante Verhaltensweisen ans Licht. Einige der hier angeführten Beispiele zeigen Limitationen auf. Den fünf verschiedenen Modellen wurde hierbei jeweils ein etwas komplexerer Text mit genau einer Lücke gegeben, um zu beobachten wie sich das Modell dabei verhält. Eine der ersten Auffälligkeiten kommt von 'bart-large', da dieses immer mit einem Punkt antwortet, wenn der maskierte Token das letzte Wort des Satzes ist.

The largest planet in our solar system is [MASK].

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | earth      | 0.188365 |
| facebook/bart-large    | .          | 1.000    |
| albert-base-v2         | jupiter    | 0.419004 |
| roberta-base           | Jupiter    | 0.344581 |
| electra-base-generator | mars       | 0.180464 |

Water boils at [MASK] degrees C°.

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | 60         | 0.041974 |
| facebook/bart-large    | a          | 0.150513 |
| albert-base-v2         | 350        | 0.022185 |
| roberta-base           | 40         | 0.030012 |
| electra-base-generator | 350        | 0.077588 |

Obwohl die Modelle normalerweise sehr gut bei einfachen Wissensfragen abschneiden, ist hier ersichtlich, dass diese bei der Frage nach dem Siedepunkt von Wasser sehr unsicher sind und diese Frage nicht richtig beantworten können. Die Planetenfrage beantworten zumindest zwei der Modelle richtig und, abgesehen von BART, antworten auch die restlichen Modelle mit Planeten.

She put the milk back in the [MASK] to keep it cold.

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | fridge     | 0.286026 |
| facebook/bart-large    | fridge     | 0.247192 |
| albert-base-v2         | fridge     | 0.351228 |
| roberta-base           | fridge     | 0.452537 |
| electra-base-generator | fridge     | 0.202559 |

She put the medicine back in the [MASK] to keep it cold.

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | bag        | 0.282260 |
| facebook/bart-large    | bottle     | 0.165283 |
| albert-base-v2         | fridge     | 0.115420 |
| roberta-base           | bottle     | 0.204963 |
| electra-base-generator | fridge     | 0.160419 |

In diesem Beispiel wurde den Modellen ein fast identischer Satz gegeben. Die Logik des Satzes wurde dabei nicht verändert, sondern einzig das Wort 'milk' durch 'medicine' ausgetauscht. Jedes der Modelle beantwortet den ersten Satz dabei korrekt. Bei dem zweiten Satz antworten jedoch nur noch zwei Modelle mit 'fridge' und das mit geringerem Score. Die anderen Modelle scheinen die Antwort zu ändern, da sie das Wort 'medicine' beispielsweise mehr mit dem Begriff 'bottle' assoziieren und weniger auf das Wort 'cold' achten. Das Beispiel zeigt auch auf, dass die Modelle das Wort 'milk' mehr mit 'fridge' verbinden, als das Wort 'medicine'.

Mary's dog is called Ludwig. Ludwigs owner is [MASK].

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | ludwig     | 0.029760 |
| facebook/bart-large    | .          | 0.987793 |
| albert-base-v2         | joyah      | 0.126011 |
| roberta-base           | David      | 0.021402 |
| electra-base-generator | ludwig     | 0.048731 |

Mary has a dog. The dogs owner is called [MASK].

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | tom        | 0.024168 |
| facebook/bart-large    | .          | 0.981445 |
| albert-base-v2         | evalle     | 0.283692 |
| roberta-base           | Mary       | 0.044635 |
| electra-base-generator | Mary       | 0.031641 |

Anhand dieser Lückentexte ist ersichtlich, dass manche Modelle in der Lage sind, Eigennamen zu verarbeiten und die Lücke sinnvoll ausfüllen können. Diese Art von Sätzen fällt in einen Teilbereich namens 'named-entity recognition'. Jedoch ist auch ersichtlich, dass die Fähigkeiten der Modelle hierbei stark vom gegebenen Satz abhängen und den meisten Modellen diese Aufgabe eher schwerfällt.

The candle melted all over the table. The [MASK] was ruined.

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | table      | 0.041987 |
| facebook/bart-large    | food       | 0.051147 |
| albert-base-v2         | candle     | 0.518183 |
| roberta-base           | candle     | 0.177422 |
| electra-base-generator | table      | 0.127574 |

The trophy doesn't fit in the shelf because it is too [MASK].

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | heavy      | 0.401578 |
| facebook/bart-large    | .          | 0.906250 |
| albert-base-v2         | tall       | 0.101796 |
| roberta-base           | heavy      | 0.191668 |
| electra-base-generator | big        | 0.166590 |

He couldn't reach the top shelf because he was too [MASK].

| Model                  | Prediction | Score    |
|------------------------|------------|----------|
| bert-base-uncased      | weak       | 0.127270 |
| facebook/bart-large    | .          | 0.986328 |
| albert-base-v2         | tired      | 0.141747 |
| roberta-base           | tall       | 0.685464 |
| electra-base-generator | big        | 0.180254 |

Unser letztes Beispiel untersucht, ob unsere Modelle simpler Logik und Begründungen folgen können. Bei dem Lückentext mit der schmelzenden Kerze wird dem Modell, vor dem maskierten Satz, ein Satz mit den Objekten 'table' und 'candle' gegeben. Hier müsste das Modell schlussfolgern, dass der Tisch durch die Kerze ruiniert werden würde. Es sagen jedoch gleich viele Modelle 'table' wie 'candle' voraus. Zudem sind sich jene Modelle, die korrekt antworteten, unsicherer als alle anderen Modelle.

Bei unserem Trophäensatz haben die meisten Modelle eine richtige Idee beziehungsweise eine richtige Begründung, warum die Trophäe nicht ins Regal passen könnte. Dennoch passen die meisten Antworten nicht richtig in die Lücke und nur ein Modell antwortete mit dem naheliegendsten Wort 'big'.

Die Schlussfolgerung hinter dem letzten Beispielsatz, scheinen die Modelle jedoch gar nicht zu verstehen. Während Wörter wie 'weak' oder 'tired' mit viel Spielraum als richtig bewertet werden können, antworten zwei Modelle mit dem kompletten Gegenteil des gesuchten Wortes.

## 2.3 Verschiedene Modelle für verschiedene Themengebiete

### 2.3.1 Einleitung

In diesem Kapitel wird untersucht, wie gut verschiedene moderne Sprachmodelle mit unterschiedlichen Arten von Texten umgehen können. Verglichen werden mehrere bekannte Modelle darunter BERT, ELECTRA, mBERT, RoBERTa, MPNet und ALBERT.

Zusätzlich wird eine menschliche Referenz als Vergleich herangezogen, um ein Gefühl dafür zu bekommen, wie weit die Modelle noch von menschlicher Leistung entfernt sind. Die Experimente werden mit verschiedenen Textsorten durchgeführt, zum Beispiel mit kurzen Texten aus Kinderbüchern, Artikeln aus der englischen Wikipedia, deutschen literarischen Texten, populären englischen Romanen sowie längeren Textpassagen. Auf diese Weise können sowohl unterschiedliche Themenbereiche als auch verschiedene Textlängen und Sprachen berücksichtigt werden. Alle Modelle werden mit derselben Aufgabe getestet: In einem Text wird ein Wort ausgeblendet, und das Modell soll dieses Wort anhand des Kontexts vorhersagen. Die Ergebnisse werden mit Hilfe von Top-1 und Top-5-Accuracy ausgewertet.

### 2.3.2 Durchführung

Das Vorgehen bestand aus zehn maskierten Sätzen, die den verschiedenen Modellen vorgelegt wurden. Im folgenden Ausschnitt wurde das Modell BERT verwendet. Diesem wurde ein Lückentext zum Thema Kinderbücher gegeben.

```
1 from transformers import pipeline
2 import pandas as pd
3
4 fill_mask = pipeline("fill-mask", model="bert-base-uncased")
5
6 kids_en = [
7     ("Little Red Riding Hood walked into the forest and met the [MASK].", "wolf"),
8     ("The king sat on his [MASK] and spoke to the people.", "throne"),
9     ("The wizard raised his [MASK] and whispered a spell.", "staff"),
10    ("The little boy lost his [MASK] in the snow.", "hat"),
11    ("The brave knight drew his [MASK] to fight the dragon.", "sword"),
12    ("The witch offered a poisoned [MASK] to the girl.", "apple"),
13    ("The princess opened the old [MASK] and looked inside.", "door"),
14    ("The giant lived at the top of the tall [MASK].", "tower"),
15    ("The fairy waved her [MASK] and the room sparkled.", "wand"),
16    ("The rabbit ran so fast that the [MASK] could not catch up.", "turtle"),
17 ]
18
19 rows = []
20 for text, gold in kids_en:
21     preds = fill_mask(text) # list of dicts
22     top5 = [p["token_str"].strip() for p in preds[:5]]
23     top1 = top5[0]
24
25     rows.append({
26         "text": text,
27         "gold": gold,
28         "top1": top1,
29         "top5": ", ".join(top5),
30         "top1_correct": (top1 == gold),
31         "top5_correct": (gold in top5),
32     })
33
34 df = pd.DataFrame(rows)
35 df
```

|   | text                                              | gold   | top1   |                                       | top5 | top1_correct | top5_correct |
|---|---------------------------------------------------|--------|--------|---------------------------------------|------|--------------|--------------|
| 0 | Little Red Riding Hood walked into the forest ... | wolf   | others | others, sheriff, rider, girl, horse   |      | False        | False        |
| 1 | The king sat on his [MASK] and spoke to the pe... | throne | throne | throne, horse, chair, knees, stool    |      | True         | True         |
| 2 | The wizard raised his [MASK] and whispered a s... | staff  | hand   | hand, head, hands, sword, voice       |      | False        | False        |
| 3 | The little boy lost his [MASK] in the snow.       | hat    | life   | life, footing, head, balance, mother  |      | False        | False        |
| 4 | The brave knight drew his [MASK] to fight the ... | sword  | sword  | sword, swords, blade, dagger, weapon  |      | True         | True         |
| 5 | The witch offered a poisoned [MASK] to the girl.  | apple  | drink  | drink, apple, cup, stone, stick       |      | False        | True         |
| 6 | The princess opened the old [MASK] and looked ... | door   | door   | door, book, box, window, chest        |      | True         | True         |
| 7 | The giant lived at the top of the tall [MASK].    | tower  | tower  | tower, mountain, building, tree, hill |      | True         | True         |
| 8 | The fairy waved her [MASK] and the room sparkled. | wand   | hand   | hand, wand, arms, hands, arm          |      | False        | True         |
| 9 | The rabbit ran so fast that the [MASK] could n... | turtle | dog    | dog, horse, wolf, cat, bird           |      | False        | False        |

Abbildung 1: Resultate von BERT zum Thema Kinderbücher

Die Daten können ausgewertet werden, in dem man betrachtet, wie viele der zehn Sätze BERT richtig beantwortet hat. Zusätzlich wurde auch untersucht, ob die richtige Antwort in den Top 5 vorhergesagten Antworten vorkommt.

```

1 top1 = df["top1_correct"].sum()
2 top5 = df["top5_correct"].sum()
3 total = len(df)
4
5 print("BERT - Children's Books Evaluation")
6 print(f"Top-1 Accuracy: {top1}/{total}")
7 print(f"Top-5 Accuracy: {top5}/{total}")
8
9 # Output:
10 # BERT - Children's Books Evaluation
11 # Top-1 Accuracy: 4/10
12 # Top-5 Accuracy: 6/10

```

Dieses Vorgehen wurde dann mit verschiedenen Modellen und mit, nach verschiedenen Themengebieten gruppierten, Sätzen wiederholt.

### 2.3.3 Ergebnisse

|         | Children |       | Wikipedia |       | German |      | Popular Books |       | Long Texts |       |
|---------|----------|-------|-----------|-------|--------|------|---------------|-------|------------|-------|
|         | T1       | T5    | T1        | T5    | T1     | T5   | T1            | T5    | T1         | T5    |
| BERT    | 4/10     | 6/10  | 6/10      | 7/10  | 0/10   | 0/10 | 3/10          | 4/10  | 2/10       | 4/10  |
| ELECTRA | 3/10     | 5/10  | 5/10      | 9/10  | 0/10   | 7/10 | 1/10          | 3/10  | 1/10       | 4/10  |
| mBERT   | 3/10     | 3/10  | 6/10      | 9/10  | 0/10   | 5/10 | 2/10          | 3/10  | 1/10       | 1/10  |
| RoBERTa | 5/10     | 7/10  | 8/10      | 8/10  | 4/10   | 7/10 | 2/10          | 3/10  | 2/10       | 7/10  |
| MPNet   | 5/10     | 7/10  | 6/10      | 8/10  | 0/10   | 0/10 | 3/10          | 3/10  | 3/10       | 4/10  |
| ALBERT  | 3/10     | 4/10  | 4/10      | 5/10  | 0/10   | 0/10 | 1/10          | 3/10  | 1/10       | 2/10  |
| Human   | 10/10    | 10/10 | 10/10     | 10/10 | 7/10   | 7/10 | 10/10         | 10/10 | 10/10      | 10/10 |

Die Ergebnisse in der Tabelle zeigen deutliche Leistungsunterschiede zwischen den getesteten Modellen sowie zwischen den verschiedenen Textarten. Insgesamt schneiden alle Modelle bei kurzen, englischen Texten - insbesondere bei Wikipedia-Artikeln und Kinderbüchersitzen- deutlich besser ab als bei längeren Texten oder bei deutschsprachigen Inhalten. Dies deutet darauf hin, dass sowohl Textstruktur als auch Sprachraum einen starken Einfluss auf die Vorhersagequalität haben. RoBERT und MPNet erzielten in mehreren Kategorien die besten Ergebnisse, insbesondere bei englischen Texten. Dies lässt sich von allem durch ihre größere

und diversere Vortrainingsdatenmenge sowie durch optimierte Trainingsstrategien erklären. BERT zeigt insgesamt stabile, aber meist etwas schwächere Leistungen, während ALBERT in fast allen Szenarien deutlich hinter den anderen Modellen zurückbleibt, was vermutlich auf seine stärkere komprimierte Modellarchitektur zurückzuführen ist. Ein besonders auffälliges Ergebnis ist die sehr schwache Performance der meisten Modelle auf deutschen Texten. Ursache hierfür sind unter anderem die komplexe Morphologie des Deutschen, häufige Komposita sowie Tokenisierungsprobleme, die dazu führen, dass viele Wörter nicht als einzelne sinnvolle Einheiten verarbeitet werden können. Selbst mBERT, das explizit für mehrere Sprachen trainiert wurde, kann diese Schwierigkeiten nur teilweise kompensieren.

#### **2.3.4 Schlussfolgerung**

Zusammenfassend lässt sich festhalten, dass moderne Sprachmodelle in strukturierten, englischsprachigen Texten gute Ergebnisse erzielen, jedoch bei komplexeren sprachlichen Strukturen und längeren Kontexten weiterhin deutliche Grenzen aufweisen. Die konstant überlegene Leistung der menschlichen Referenz unterstreicht, dass aktueller Sprachmodellen noch ein tiefgehendes semantisches Verständnis fehlt.



### 3 Finetune Beispiel mit BERT

Im folgenden Kapitel werden wir uns nun mit einem Anwendungsbeispiel beschäftigen welches ein BERT Modell nutzt um Informationen für Nutzer bereitzustellen.

Wir wollen nun BERT nutzen um einem Benutzer die Möglichkeit zu geben durch Natürliche Fragen Informationen über eine Fiktive Firma zu erhalten. Daher betrachten wir nun einmal unser Trainingsset<sup>1</sup>

#### 3.1 Dataset

Wir betrachten die Fiktive Firma *Example Tech* mit Standorten in Berlin, München und Hamburg. Wir haben nun eine Liste von Mitarbeitern welche in bestimmten Positionen an bestimmten Orten arbeiten

```
1  {
2      "name": "Alice",
3      "role": "CEO",
4      "location": "Berlin",
5      "email": "ceo@exampletech.com"
6  },
7  {
8      "name": "Carol",
9      "role": "CFO",
10     "location": "Munich",
11     "email": "cfo@exampletech.com"
12 },
13 {
14     "name": "Lily",
15     "role": "Accountant",
16     "location": "Hamburg",
17     "email": "accounting.hamburg@exampletech.com"
18 },
19 ....
```

Insgesamt gibt es 24 Personen mit verschiedenen Jobs, Namen und Arbeitsorten. Weiters bietet das Unternehmen noch verschiedene Produkte an

```
1  {
2      "name": "AlphaPhone",
3      "category": "Electronics",
4      "price": 699
5  },
6  {
7      "name": "OrionERP",
8      "category": "Software",
9      "price": 99
10 },
11 {
12     "name": "LightPanel",
13     "category": "Furniture",
14     "price": 149
15 },
16
```

<sup>1</sup>Das Datenset und der Code wurde teils mittels ChatGPT erstellt und anschließend von menschlicher Hand korrigiert

```
17 {
18     "name": "TechSupport+",
19     "category": "Service",
20     "price": 39
21 },
22 ....
```

Es gibt gesamt 19 Produkte.

Nun können wir uns Fragen wie wir dieses Datenset in eine Form überführen mit welcher wir BERT trainieren können. Die Idee war nun dies mittels Natürlicher Fragen über das Datenset zu machen. Wir trainieren also auf Text wie

*Alice is the CEO working in the Berlin Office*

oder

*Alice's email address is ceo@exampletech.com*

und auch Fragen wie

*Where is Alice? In the Berlin Office.*

Selbes machen wir für die Produkte. Würden wir nun diese Fragen einfach dem Tokenizer geben hätten wir jedoch ein Problem. Denn z.B. CEO ist *Ein* Token jedoch z.B. Sales Manager *zwei* Token und eine EMail adresse wird sogar in bis zu *fünf* Token aufgespaltet. Daher unsere Lösung, wir fügen einfach einige neue Token hinzu. Dies lässt sich mittels Transformerers sehr einfach machen.

```
1 emails = [
2     e["email"] for e in json.load(open("dataset.json"))["employees"]
3 ]
4
5
6 jobs = [
7     e["role"] for e in json.load(open("dataset.json"))["employees"]
8 ]
9
10
11 products = [
12     e["name"] for e in json.load(open("dataset.json"))["products"]
13 ]
14
15
16 prices = [
17     str(e["price"]) for e in json.load(open("dataset.json"))["products"]
18 ]
19
20 companyName = [json.load(open("dataset.json"))["company"]["name"]]
21 ]
22
23 tokenizer.add_tokens(emails)
24 tokenizer.add_tokens(jobs)
25 tokenizer.add_tokens(products)
26 tokenizer.add_tokens(prices)
27 tokenizer.add_tokens(companyName)
28
29 # WICHTIG Model auf neues Vokabular anpassen
30 model.resize_token_embeddings(len(tokenizer))
```

### 3.2 Training

Das Training wird lokal mittels PyTorch und Transformerers durchgeführt. Der genutzte PC hat die folgenden Spezifikationen

|                |                              |
|----------------|------------------------------|
| CPU            | AMD Ryzen 7 7700X            |
| GPU            | NVIDIA GeForce RTX 3060 12GB |
| RAM            | 32 GB - DDR5 4800 MT/s       |
| Betriebssystem | Arch Linux - Kernel 6.18.2   |
| CUDA           | 12.6                         |

Die Transformerers Bibliothek hat glücklicherweise bereits eine Implementation für BERT somit können wir diese einfach nutzen. Zunächst importieren wir den Tokenizer und das Model sowie den Trainer.

```
1 import torch
2 from transformers import (
3     BertTokenizer,
4     BertForMaskedLM,
5     DataCollatorForLanguageModeling,
6     Trainer,
7     TrainingArguments,
8     EarlyStoppingCallback)
```

damit ist schon fast die Hälfte des Coding Aufwands geschafft.

Unsere nächste Überlegung ist nun welches Model wir nutzen wollen zum Trainieren. Unsere Wahl fiel auf *bert-base-uncased*. Dieses ist ein Basis BERT Model welche nicht zwischen Groß- und Kleinschreibung unterscheidet. Wie bereits vorher diskutiert wurden dem Tokenizer die EMail-Adressen, Jobs und Produkte als Token hinzugefügt damit wir uns nicht über die genaue Anzahl an Mask gedanken machen müssen. Um nun das Model und den Tokenizer zu laden können wir einfach Transformerers nutzen

```
1 model = BertForMaskedLM.from_pretrained("bert-base-uncased")
2 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

Transformerers wird uns nun automatisch das Model herunterladen und lokal speichern. Dies kann beim ersten Start zu einiger Wartezeit führen je nach Internetvertrag.

Wir wollen nun den Trainer vorbereiten. Hierzu nutzen wir die *TrainingArguments*. Wir nutzen hierbei die folgenden Argumente:

| Argument                    | Wert               | Beschreibung                                                                                                                                                                         |
|-----------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| num_train_epochs            | 500                | Anzahl der Trainingsdurchläufe (Epochen). Eine Epoche bedeutet, dass das gesamte Trainingsdatenset einmal vollständig verarbeitet wurde.                                             |
| per_device_train_batch_size | 16                 | Anzahl der Trainingsbeispiele, die gleichzeitig (in einem Batch) verarbeitet werden. Eine größere Batch-Größe benötigt mehr Speicher, kann aber das Training stabiler machen.        |
| learning_rate               | $5 \times 10^{-5}$ | Lernrate, also wie stark die Modellparameter pro Trainingsschritt angepasst werden. Zu große Werte können das Training instabil machen, zu kleine führen zu sehr langsamem Lernen.   |
| save_steps                  | 3000               | Anzahl der Trainingsschritte zwischen dem Speichern von Modell-Zwischenständen (Checkpoints).                                                                                        |
| save_total_limit            | 2                  | Maximale Anzahl gespeicherter Checkpoints. Ältere Zwischenstände werden automatisch gelöscht, um Speicherplatz zu sparen.                                                            |
| logging_steps               | 100                | Gibt an, nach wie vielen Trainingsschritten Trainingsmetriken (z. B. Loss) berechnet und ausgegeben werden.                                                                          |
| lr_scheduler_type           | linear             | Art des Lernraten-Schedulers. Bei linear wird die Lernrate nach der Aufwärmphase schrittweise bis auf null reduziert.                                                                |
| warmup_ratio                | 0.1                | Anteil der gesamten Trainingsschritte, in denen die Lernrate von null auf den maximalen Wert erhöht wird (Warmup-Phase). Dies hilft, ein instabiles Training zu Beginn zu vermeiden. |

[1] Nun können wir also mit diesen Argumenten einen Trainer erstellen.

```

1 training_args = TrainingArguments(
2     #Arguments
3 )
4
5 trainer = Trainer(
6     model=model,
7     args=training_args,
8     train_dataset=dataset,
9     data_collator=data_collator,
10 )

```

Nun sind wir endlich bereit den Trainer zu starten. Dies ist simple und einfach. Anschließend können wir das Model noch speichern damit wir es später verwenden können.

```

1 trainer.train()
2
3 model.save_pretrained("./bert-company")
4 tokenizer.save_pretrained("./bert-company")

```

### 3.3 Auswertung

Das Training dauert ca. 15 Minuten auf dem oben angeführten PC. Nach Abschluss ist nun die Frage wie Werten wir den Finetune aus? Hierzu wurden verschiedene Fragen für jedes gelernte Datum gestellt, welche nicht zum Training genutzt wurden. Die Fragen lauten wie folgt

name works in [MASK].

The email address [MASK] belongs to name.

Who is role in location? [MASK].

product-name sales for [MASK] euros.

Diese Fragen wurden nun für jede Person/Produkt ausgewertet und BERT gegeben. Das Ergebnis lautet wie folgt:

Correct: 98/101

Accuracy: 97.02970297029702%

Von 101 Fragen wurden also 98 richtig beantwortet. Welche führten nun zu Probleme?

Input: Who is CFO in Munich? [MASK].

Output: who is cfo in munich? cfo@exampletech.com.

Correct: who is cfo in munich? carol.

Input: Who is Marketing Specialist in Munich? [MASK].

Output: who is marketing specialist in munich? marketing.munich@exampletech.com.

Correct: who is marketing specialist in munich? nathan.

Input: Who is Customer Support in Hamburg? [MASK].

Output: who is customer support in hamburg? customer@exampletech.com.

Correct: who is customer support in hamburg? lana.

Wir sehen das in allen drei Fällen der Name mit einer Email verwechselt wurde. Betrachten wir für jede dieser Fragen die 5 Wahrscheinlichsten Token so sehen wir

Input: Who is CFO in Munich? [MASK].

cfo@exampletech.com (p=0.6099)

carol (p=0.3184)

cfo (p=0.0324)

munich (p=0.0098)

hr.munich@exampletech.com (p=0.0063)

Input: Who is Marketing Specialist in Munich? [MASK].

marketing.munich@exampletech.com (p=0.4459)

nathan (p=0.3087)

marketing specialist (p=0.0701)

hr.munich@exampletech.com (p=0.0401)

munich (p=0.0349)

Input: Who is Customer Support in Hamburg? [MASK].

customer@exampletech.com (p=0.6026)

lana (p=0.2535)

customer support (p=0.0983)

hamburg (p=0.0061)

it@exampletech.com (p=0.0051)

Jedes Mal ist die richtige Antwort die zweit Wahrscheinlichste und das mit einem nicht zu vernachlässigen Prozentsatz. Wir können nun einmal betrachten wie es bei den richtigen Antworten aussieht. Die Fragen mit den geringsten Wahrscheinlichkeitswerten sind die folgenden

57.04% | Who is HR Assistant in Hamburg? [MASK].  
 59.45% | Who is Support Lead in Berlin? [MASK].  
 64.54% | Who is HR Manager in Berlin? [MASK].  
 71.19% | Who is HR Assistant in Munich? [MASK].  
 78.07% | Who is IT Director in Hamburg? [MASK].

Generell lässt sich die Wahrscheinlichkeit(Confidence) im folgenden Diagramm darstellen.

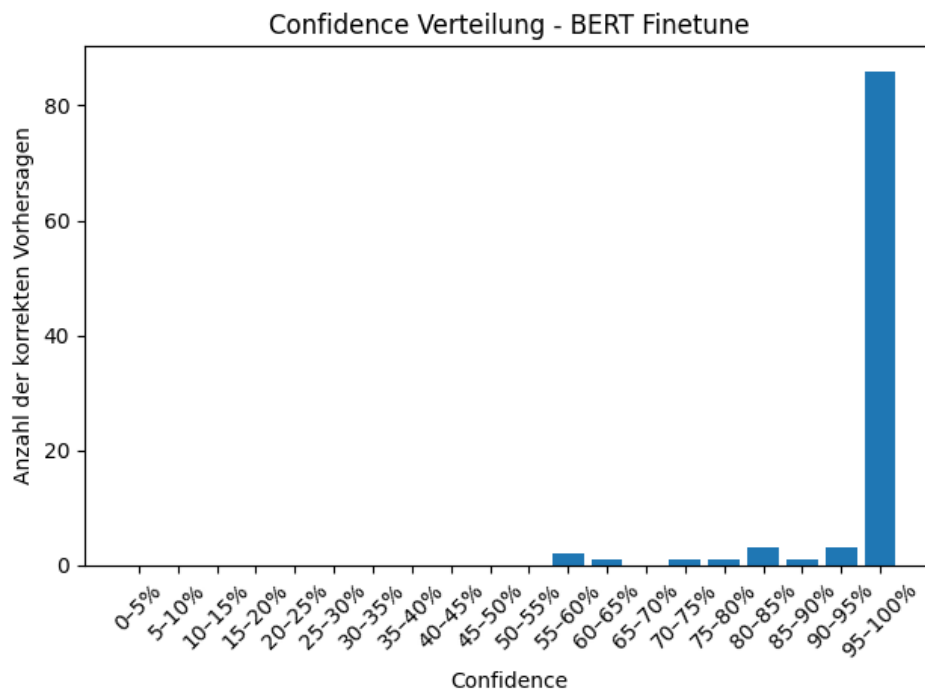


Abbildung 2: Confidence des BERT-Finetune

Wir sehen also bei einem Großteil der Vorhersagen ist sich das Modell sehr sicher. 86 von 98 Vorhersagen haben eine Sicherheit von mehr als 95 % und lediglich 5 eine kleiner als 80 %. Dennoch sehen wir das Modell ist vor allem bei Fragen etwas unsicher. Somit wäre ein Training mit etwas mehr Fragen sicherlich hilfreich um die Sicherheit weiter zu stärken.

### 3.4 Informationsverlust

Wir wollen uns nun zum Abschluss dieses Kapitels noch anschauen, was eigentlich der TradeOff unseres Trainings war. Dazu Vergleichen wir die Performance des Finetune Modells mit dem Originalmodell. Wir nutzen dazu dieselben Beispiele wie in Kapitel 2.

|          | Children |      | Wikipedia |      | German |      | Popular Books |      | Long Texts |      |
|----------|----------|------|-----------|------|--------|------|---------------|------|------------|------|
|          | T1       | T5   | T1        | T5   | T1     | T5   | T1            | T5   | T1         | T5   |
| BERT     | 4/10     | 6/10 | 6/10      | 7/10 | 0/10   | 0/10 | 3/10          | 4/10 | 2/10       | 4/10 |
| FINETUNE | 2/10     | 2/10 | 2/10      | 2/10 | 0/10   | 0/10 | 0/10          | 0/10 | 0/10       | 0/10 |

Wir sehen also wir haben extrem an Informationen verloren durch das Training. Dieses Modell wäre als General Purpose Modell komplett ungeeignet nun.

## 4 Korrektur von Lückentexten

In diesem Teil wollten wir einem Modell einen ausgefüllten Lückentext geben, damit es ihn korrigiert: Wurde der Text richtig oder falsch ausgefüllt?

### 4.1 Erster Versuch mit BERT

#### 4.1.1 Datenset für Korrektur

Zunächst wurde hier ein öffentliches Datenset "Cloze-test-with-Bert"<sup>2</sup> verwendet. Dieses besteht aus einem Test- und einem Trainingsset, wobei das Testset 1.267 und das Trainingsset 40.398 Sätze mit jeweils zwei Antwortmöglichkeiten beinhaltet. Eine Datei aus dem Datenset sieht z.B. so aus:

```
1 {
2   "qID": "3FBEFUUYRMJCQIM5XJ0W8CIQYEGA6Z-1",
3
4   "sentence": "Jane received a pet tortoise and an aquarium as a
5   birthday gift, but the _ was too small.",
6
7   "option1": "aquarium",
8   "option2": "tortoise",
9
10  "answer": "1"
11 }
```

Um das Datenset gut benutzen zu können haben wir es mithilfe Python transformiert:

```
1 input_file = "./test.jsonl"
2 output_file = "./testnew.jsonl" #oder "./train.jsonl" und "./trainnew.jsonl"
3
4 with open(input_file, "r", encoding="utf-8") as fin, \
5     open(output_file, "w", encoding="utf-8") as fout:
6
7     for line in fin:
8         data = json.loads(line)
9
10        sentence = data["sentence"]
11        masked_sentence = sentence.replace("_", "[MASK]") #da BERT [MASK] braucht
12        opt1 = data["option1"]
13        opt2 = data["option2"]
14        correct = data["answer"] # "1" oder "2"
15
16        filled_1 = sentence.replace("_", opt1) # Option 1 einsetzen
17        example_1 = {
18            "qID": data["qID"],
19            "unfilled_sentence": masked_sentence,
20            "filled_sentence": filled_1,
21            "chosen_option": opt1,
22            "is_correct": correct == "1"
23        }
24
25        filled_2 = sentence.replace("_", opt2) # Option 2 einsetzen
26        example_2 = {
27            "qID": data["qID"],
```

<sup>2</sup><https://github.com/TheMaxi7/Cloze-test-with-BERT>

```

28         "unfilled_sentence": masked_sentence,
29         "filled_sentence": filled_2,
30         "chosen_option": opt2,
31         "is_correct": correct == "2"
32     }
33
34     fout.write(json.dumps(example_1, ensure_ascii=False) + "\n")
35     fout.write(json.dumps(example_2, ensure_ascii=False) + "\n")

```

Dieser Code bringt das Datenset in eine Form, die für unsere Anwendung besser ist. Jede Datei wird aufgeteilt in zwei Dateien. Die obere Beispieldatei wird auf zwei Dateien aufgeteilt:

```

1  {
2      "qID": "3FBEFUUYRMJCQIM5XJOW8CIQYEGA6Z-1",
3
4      "unfilled_sentence": "Jane received a pet tortoise and an aquarium
as a birthday gift, but the [MASK] was too small.",
5
6      "filled_sentence": "Jane received a pet tortoise and an aquarium as
a birthday gift, but the aquarium was too small.",
7
8      "chosen_option": "aquarium",
9
10     "is_correct": true
11 }
12
13 {
14     "qID": "3FBEFUUYRMJCQIM5XJOW8CIQYEGA6Z-1",
15
16     "unfilled_sentence": "Jane received a pet tortoise and an aquarium
as a birthday gift, but the [MASK] was too small.",
17
18     "filled_sentence": "Jane received a pet tortoise and an aquarium as
a birthday gift, but the tortoise was too small.",
19
20     "chosen_option": "tortoise",
21
22     "is_correct": false
23 }

```

#### 4.1.2 Korrektur mit BERT

Zuerst versuchen wir den Text mit einem vortrainierten BERT-Modell zu korrigieren. Wir machen dies, indem wir die Funktion `token_probability` definieren, die einen Satz mit `[MASK]`-Token tokenized, die Position des `[MASK]`-Tokens findet, Logits an der Maskenposition auswählt und dann die Logits mittels Softmax in Wahrscheinlichkeiten umwandelt. Das eingesetzte Wort wird von ihr auch tokenized, wobei ein `ValueError` ausgegeben wird, falls das Wort aus mehr als einem Token besteht, um sie später einfach überspringen zu können, da BERT nur einzelne Tokens bewerten kann. Die Funktion gibt am Ende die bedingte Wahrscheinlichkeit aus, dass das Modell den Token vom eingesetzten Wort an der Maskenposition vorhersagt. Danach wird für jeden Satz in jeder Zeile der jsonl-Datei mithilfe von `token_probability` die bedingte Wahrscheinlichkeit berechnet, außer sie wird übersprungen, da das eingesetzte Wort aus mehreren Tokens besteht. Schließlich geben wir die Anzahl der verwendeten und übersprungenen Sätze an.



Um einen guten Threshold zu finden, ab welcher Wahrscheinlichkeit wir sagen, dass der Satz korrekt ausgefüllt wurde, nehmen wir 300 Werte zwischen der kleinsten und größten Wahrscheinlichkeit. Für jeden dieser Wahrscheinlichkeitswerte schauen wir uns an welche Wörter eine höhere Wahrscheinlichkeit haben und überprüfen wie viele davon korrekt sind. Der Threshold mit den meisten richtigen Vorhersagen wird als bester Threshold ausgegeben, zusammen mit der Accuracy unter diesem Threshold.

```
1 import json
2 import torch
3 import numpy as np
4 from transformers import BertTokenizer, BertForMaskedLM
5 import torch.nn.functional as torchF
6
7 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
8 model = BertForMaskedLM.from_pretrained("bert-base-uncased")
9 model.eval()
10
11 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
12 model.to(device)
13
14 print("Running on:", device)
15
16 input_file = "./trainnew.jsonl"
17
18 scores = [] # p(w | context)
19 labels = [] # is_correct
20 skipped = 0
21
22
23 def token_probability(masked_sentence, word):
24     inputs = tokenizer(masked_sentence, return_tensors="pt").to(device)
25
26     mask_index = torch.where(
27         inputs["input_ids"] == tokenizer.mask_token_id
28     )[1]
29
30     with torch.no_grad():
31         outputs = model(**inputs)
32         logits = outputs.logits
33
34     mask_logits = logits[0, mask_index, :]
35     probs = torchF.softmax(mask_logits, dim=-1)
36
37     token_ids = tokenizer.convert_tokens_to_ids(
38         tokenizer.tokenize(word)
39     )
40
41     if len(token_ids) != 1:
42         raise ValueError
43
44     return probs[0, token_ids[0]].item()
45
46
47 #Scores sammeln
48
49 with open(input_file, "r", encoding="utf-8") as fin:
50     for line in fin:
51         data = json.loads(line)
52
53         masked_sentence = data["unfilled_sentence"]
54         chosen_option = data["chosen_option"]
55         is_correct_label = data["is_correct"]
56
57         try:
```

```

58         p = token_probability(masked_sentence, chosen_option)
59     except ValueError:
60         skipped += 1
61         continue
62
63     scores.append(p)
64     labels.append(is_correct_label)
65
66 scores = np.array(scores)
67 labels = np.array(labels)
68
69 print(f"Verwendete Samples: {len(scores)}")
70 print(f"Uebersprungene Samples: {skipped}")
71
72 #Optimalen Threshold finden
73
74 best_acc = 0.0
75 best_threshold = 0.0
76 thresholdcount = 1000000
77
78 for t in np.linspace(scores.min(), scores.max(), thresholdcount):
79     preds = scores > t
80     acc = (preds == labels).mean()
81     if acc > best_acc:
82         best_acc = acc
83         best_threshold = t
84
85 print(f"Anzahl der getesteten Thresholds: {thresholdcount}")
86 print(f"Best Threshold: {best_threshold:.6f}")
87 print(f"Best Accuracy: {best_acc * 100:.2f}%")

```

Der Code gibt auf dem "Trainingsset" folgendes Resultat aus:

```

Running on: cuda
Verwendete Samples: 74644
Übersprungene Samples: 6152
Anzahl der getesteten Thresholds: 1000000
Best Threshold: 0.056973
Best Accuracy: 52.65%

```

Man sieht, dass die Accuracy auch mit dem besten Threshold nur etwas mehr als 50% beträgt, d.h. dass das Modell kaum besser als Raten ist. Das kann mehrere Gründe haben. Einerseits ist das Datenset im Nachhinein betrachtet nicht ideal für diese Aufgabe:

Viele Sätze basieren auf Logik und ein großer Teil der eingesetzten Wörter sind Namen. Andererseits sind die bedingten Wahrscheinlichkeiten der einzelnen Wörter sehr klein, da BERT ein großes Vokabular hat. Das macht es schwer einen vernünftigen Threshold zu finden, selbst mit hoher Anzahl von Thresholds.

## 4.2 Korrektur auf CLOTH Datenset

Da das vorherige Datenset nicht ideal war, versuchen wir es nun mit dem CLOTH Datenset<sup>3</sup>, das aus 7.131 Lückentextpassagen mit 99.433 Lücken besteht. Die Daten stammen aus Englischtests an Schulen in China. Jede Passage im Datenset ist eine JSON-Datei, die folgendes enthält:

```

article: Ein String mit mehreren Lücken, die mit _ markiert sind.
options: Eine Liste von Listen mit vier Antwortmöglichkeiten für jede Lücke.
answers: Eine Liste, die die richtige Antwort für jede Lücke mit "A", "B", "C" oder
        "D" angibt.
source: Eine eindeutige ID der Passage.

```

<sup>3</sup><https://www.cs.cmu.edu/glai1/data/cloth/>

Im folgenden Code laden wir 100 Passagen aus dem Validierungsset des CLOTH-Datensets. Für jede Lücke in dieser Passage setzen wir die verschiedenen Antwortmöglichkeiten ein und lassen die bedingten Scores dieser berechnen. Anschließend suchen wir wieder einen guten Threshold, um viele richtige Vorhersagen zu bekommen.

```
1 import glob
2 import json
3 import numpy as np
4
5 letter_to_index = {"A":0, "B":1, "C":2, "D":3}
6
7 def build_masked_sentence(parts, blank_index):
8     return parts[blank_index] + " [MASK] " + parts[blank_index + 1]
9
10 def score_option(masked_sentence, option):
11     #return score_sentence(masked_sentence.replace("[MASK]", option))
12
13     inputs = tokenizer(masked_sentence, return_tensors="pt")
14     mask_index = torch.where(inputs["input_ids"][0] == tokenizer.mask_token_id)[0]
15     with torch.no_grad():
16         logits = model(**inputs).logits
17     option_id = tokenizer.convert_tokens_to_ids(option)
18     return logits[0, mask_index, option_id].item()
19
20 def predict_answers2(article, options, correct_indices):
21     parts = article.split("_")
22     test = []
23
24     for i, opts in enumerate(options):
25         masked_sentence = build_masked_sentence(parts, i)
26         #print(masked_sentence)
27         #print(list(enumerate(opts)))
28         scores = [[score_option(masked_sentence, opt), correct_indices[i]==j] for j,
29 opt in enumerate(opts)]
30         for x in scores:
31             test.append(x)
32     return test
33
34 all_files = glob.glob("/kaggle/input/clothdata/CLOTH/valid/middle/middle24*.json")
35 total_correct = 0
36 total_questions = 0
37
38 results = []
39 scores = []
40 labels = []
41
42 for file_path in all_files:
43     with open(file_path, "r", encoding="utf-8") as f:
44         data = json.load(f)
45
46         article = data["article"]
47         options = data["options"]
48         answers = data["answers"]
49
50         correct_indices = [letter_to_index[a] for a in answers]
51
52         predicted = predict_answers2(article, options, correct_indices)
53
54         for i in predicted:
55             scores.append(i[0])
56             labels.append(i[1])
57
```

```
58 scores = np.array(scores)
59 labels = np.array(labels)
60
61 print(f"Verwendete Samples: {len(scores)}")
62
63 best_acc = 0.0
64 best_threshold = 0.0
65 thresholdcount = 300
66
67 for t in np.linspace(scores.min(), scores.max(), thresholdcount):
68     preds = scores > t
69     acc = (preds == labels).mean()
70     if acc > best_acc:
71         best_acc = acc
72         best_threshold = t
73
74 print(f"Processed {len(all_files)} files")
75 print(f"Anzahl der getesteten Thresholds: {thresholdcount}")
76 print(f"Best Threshold: {best_threshold:.6f}")
77 print(f"Best Accuracy: {best_acc * 100:.2f}%")
```

Wir bekommen den Output:

```
Verwendete Samples: 3644
Processed 100 files
Anzahl der getesteten Thresholds: 300
Best Threshold: 10.734425
Best Accuracy: 83.70%
```

Man sieht, es funktioniert auf diesem Datenset deutlich besser als beim Ersten. Wenn man hier raten würde, betrage die Accuracy ca. 50% und falls man immer auf "falsch ausgefüllt" tippen würde 75% . (Da wir immer alle vier Antwortmöglichkeiten eingesetzt haben, also pro Lücke jeweils immer drei falsche und eine richtige Antwort korrigiert haben.)

### 4.3 Vergleich von verschiedenen Methoden zum Korrigieren von Lückentexten

## Arbeitsaufteilung

**Krystian Gluc** Abschnitt 4 - Korrektur von Lückentexten

**Cristian Grigore** Abschnitt 2.3 - Verschiedene Modelle für verschiedene Themengebiete

**Dominik Berghammer** Abschnitt 1.1-2.2 - Initiale Evaluierung der drei Themengebiete

**Adrian Linder** Abschnitt 3 - Finetuning von BERT

## Literatur

- [1] Transformers Dokumentation. <https://huggingface.co/docs/transformers/v5.0.0/en/index>.  
Aufgerufen 30.01.2026.
- [2] Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Well-read students learn better: On the importance of pre-training compact models. *arXiv preprint arXiv:1908.08962v2*, 2019.